

hint²: Hierarchical World Models for Inference-Time Temporal Logic Guidance

Moritz Zoellner, Anastasios Manganaris, Ahmed H. Qureshi, and Rohan Paleja

Department of Computer Science, Purdue University
{zoellner, amangana, ahqureshi, rpaleja}@purdue.edu

Abstract:

A central goal of robot learning is to enable robots to execute rich instructions specified at runtime. Large-scale language-conditioned policies have made substantial progress toward this goal, yet still struggle with temporal structure and safety constraints. Linear Temporal Logic (LTL) provides a powerful language to express complex, non-Markovian instructions. However, guiding learned manipulation policies toward LTL satisfaction remains challenging because modern policies generate short-horizon action chunks and replan in closed loop, while almost all LTL specifications are evaluated over long-horizon trajectories. In this paper, we introduce hint², a method for guiding short-horizon policies toward satisfying complex LTL specifications at inference time using hierarchical world models. Our key idea is to derive two separate guidance objectives using each world model’s abstraction level. A high-level model predicts future action-induced transitions in task-relevant atomic propositions to guide progress through the LTL automaton, while a low-level dynamics model predicts immediate state evolution for accurate local safety guidance. Our results show that hint² overcomes the limitations of current LTL-guided diffusion methods, outperforms existing inference-time steering methods in CALVIN, and successfully completes instructions with complex liveness and safety constraints more elegantly than language-conditioned alternatives. Finally, we demonstrate that hint² can handle complex instructions on a real UR5e manipulator. Videos are available on [our project page](#).

Keywords: Hierarchical World Models, Inference-Time Policy Steering, Generalist Robot Policies, Temporal Logic

1 Introduction

Large-scale imitation learning has made remarkable progress toward robots that can execute language instructions specified at inference time [1, 2], showing impressive generalization in both language semantics [3] and physical behaviors [4]. However, these generalist robot policies continue to struggle with long-horizon, non-Markovian instructions [5, 6] and safety constraints specified at runtime [7, 8]. Learning this behavior by simply providing more language-conditioned demonstrations remains challenging because the amount of required training data grows combinatorially with task horizon [9] and deployment-time safety preferences are often unknown during training. Temporal logic (TL) provides a structured framework for expressing temporal and spatial constraints [10, 11, 12], while remaining closely aligned with natural language through language-to-logic interfaces [13, 14]. Enabling modern learned policies to execute TL specifications would address two central limitations of language-conditioned policies: implicit temporal progress tracking and the lack of a grounded interface for runtime safety constraints.

However, executing general TL specifications with learned policies remains limited. One approach is to condition a policy directly on the specification during training, but this inherits the scaling limitations of language conditioning [15]. Inference-time guidance is attractive because it leaves

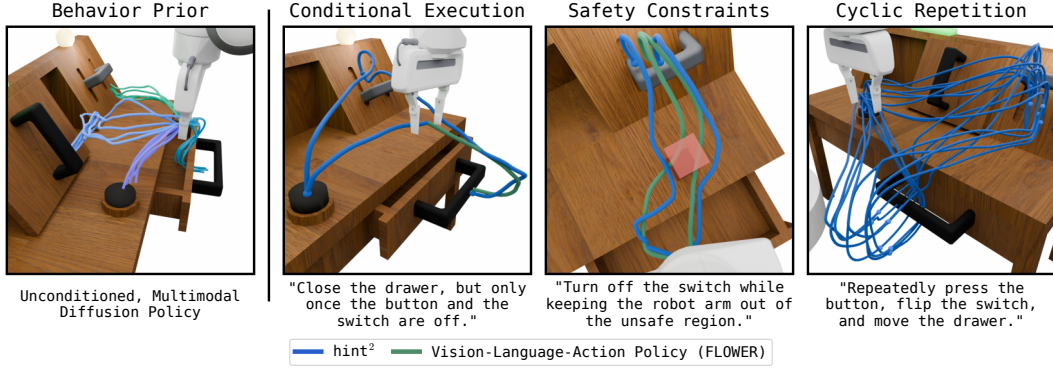


Figure 1: **Our method** hint^2 can guide a diffusion policy based on TL constraints specified at inference time. In the CALVIN environment, hint^2 executes complex long-horizon instructions that a state-of-the-art vision-language-action policy trained on the same data fails to complete.

the learned policy unchanged and instead modifies how actions are selected from its distribution. Recent work has shown that such guidance is feasible in complex settings by steering diffusion policies toward simple objectives such as goal images or human-supplied keypoints [16, 17]. TL poses a richer objective that is more challenging for inference-time guidance. Current TL-guided diffusion approaches generate state-action trajectories for the full task and guide them using differentiable robustness values [18, 19]. This fails in complex domains, where predicting a state trajectory over the full task requires long-horizon world-model rollouts that face compounding errors as a fundamental limitation [20, 21]. Additionally, modern manipulation policies generate only short action chunks and replan in a closed loop [22]. This creates a fundamental horizon mismatch for inference-time TL guidance: evaluating robustness over states in this short horizon severely restricts the supported specifications and effectively reduces guidance to safety filtering, since the truth value of many TL specifications may only be decided many steps into the future [23, 24].

We address this mismatch by introducing hint^2 , a method for guiding short-horizon diffusion policies toward long-horizon TL satisfaction. hint^2 learns two world models at different abstraction levels. A high-level world model predicts how an action chunk induces future transitions in task-relevant atomic propositions, enabling guidance toward progress through the Linear Temporal Logic (LTL) automaton. A low-level world model predicts the immediate state consequences of those actions, enabling Signal Temporal Logic (STL) robustness guidance for safety constraints where precise geometry matters. Together, these world models allow hint^2 to select action chunks from an unconditioned, multimodal diffusion policy that satisfy both long-horizon planning objectives and local safety requirements specified at inference-time. We conduct a systematic evaluation of how hint^2 compares to existing guidance methods in their intended settings. Figure 1 illustrates the capabilities enabled by hint^2 in the CALVIN [25] environment, where it executes complex instructions that language-conditioned counterparts [26] fail to complete.

2 Related Work

Temporal logic for robot task specification. TL specifications have long been used to express non-Markovian robot behaviors [10] and can be decomposed into *liveness constraints*, which require desired events to eventually occur, and *safety constraints*, which require undesired events to never occur [27, 28]. Classical approaches typically compose an abstraction of the robot-environment dynamics with an automaton encoding an LTL specification, then define a controller over the resulting product system [29, 30]. This automaton-based view has also shaped reinforcement learning for non-Markovian objectives, where automaton progress augments the agent state to enable Markovian policy learning [31, 32, 33, 34]. A complementary line of work extends TL objectives beyond discrete automata through STL robustness, differentiable logic losses, and model-based controllers for continuous systems [35, 36, 37, 38, 39]. Our work draws on both views by using automata for long-horizon task progress and differentiable robustness for local safety guidance.

Guiding learned policies at inference time. The dominant approach to controlling learned robot policies is training-time conditioning, most commonly through language conditioning [22, 3, 1, 40], but also through conditioning on formal specifications [15, 41]. Inference-time guidance instead steers a pretrained policy toward objectives not necessarily covered during training, for example by biasing diffusion denoising [42, 43] or selecting sampled actions that score best under a learned objective [17]. In robotics, this has enabled steering toward goal observations, human preferences, and safety objectives [44, 16, 45]. TL-based guidance has also been explored by evaluating specifications over state-action trajectories [46, 18, 19, 47], but remains limited to settings where the full trajectory can be generated and evaluated in one shot. Our method uses the same guidance mechanisms, but derives an objective that steers short-horizon policies toward long-horizon TL satisfaction.

World models for long-horizon robot behavior. World models have become increasingly effective at predicting action-induced state changes in complex control domains [4, 21]. Combined with differentiable robustness, such predictions can guide policies toward local STL satisfaction [36, 19], but remain fundamentally limited for long-horizon evaluation due to compounding rollout errors. Recent work addresses this limitation by moving long-horizon reasoning to a higher abstraction level using hierarchical world models that guide lower-level controllers [48, 49, 50]. These abstractions extend the planning horizon, but are either defined through task-specific planning domains, which fixes the supported structure ahead of time, or learned as latent states, which reintroduces the burden of discovering the right planning abstraction from data. `hint`² instead uses the LTL automaton as a specification-induced abstraction, which compactly captures temporal progress, while decoupling high-level planning from the fixed behavior policy used for execution.

3 Preliminaries and Problem Formulation

Temporal Logic. LTL [51] is a propositional logic for reasoning over time. LTL formulas are defined with respect to a set of atomic propositions AP , and more complex propositions are built from simpler expressions, including $p \in AP$, using the standard boolean operators ($\neg$, $\wedge$, $\vee$) and the temporal operators “next” (X), “eventually” (F), “always” (G), and “until” (U). We refer to Baier and Katoen [11] for a full overview of LTL. Our method utilizes a quantitative semantics for LTL similar to those used for STL [12] but without support for time intervals. For a Markov Decision Process (MDP) $\mathcal{M}$, with state space S and action space A , the quantitative semantics define a robustness function $\rho : S^\omega \times \Phi \rightarrow \mathbb{R}$ that produces a positive or negative value indicating the satisfaction or violation of an LTL formula $\phi \in \Phi$ by a (possibly infinite) trajectory $\tau \in S^\omega$.

Automata. Our method specifically focuses on LTL formulas in the recurrence class [52], which can be translated into abstract machines called Deterministic Büchi Automata (DBAs) [11]. A DBA obtained from an LTL formula ϕ is a tuple $\mathcal{A}_\phi = (Q, \Sigma, \delta, q_0, F)$, consisting of a set of states Q , the alphabet $\Sigma = 2^{AP}$, a transition function $\delta : Q \times \Sigma \rightarrow Q$, an initial state $q_0 \in Q$, and a set of accepting states $F \subseteq Q$. A labeling function $L : S \rightarrow 2^{AP}$ is obtained by evaluating each atomic proposition $p \in AP$ at state $s \in S$, and an infinite sequence of labels $L(s_0), L(s_1), \dots$ induces a sequence of automaton states $q_0, q_1, \dots$ via δ , satisfying ϕ if and only if the sequence visits F infinitely often. Augmenting an MDP $\mathcal{M}$ with the states Q and transition function $\delta(q_t, L(s_{t+1}))$ of an automaton $\mathcal{A}$ yields a product MDP, denoted by $\mathcal{M}_\mathcal{A}$, in which a minimal Markovian memory of task-relevant history is provided by the automaton state q_t . For our method, we define an automaton potential vector $v \in \mathbb{R}^{|Q|}$ [30] with the q -th element $v_q = \max_{q' \in F} \gamma^{d_\mathcal{A}(q, q')}$, where $\gamma \in [0, 1]$ is a discount factor and $d_\mathcal{A}(q, q')$ is the unweighted shortest-path distance from q to q' .

Problem Formulation. For an MDP $\mathcal{M}$, let $\pi(a_{t:t+H}|s_t)$ be a diffusion policy, with action horizon H , trained to imitate an expert policy $\pi^*(a_{t:t+H}|s_t)$ [22]. We wish to obtain a policy $\hat{\pi}(a_{t:t+H}|s_t, q_t, \phi)$ that minimizes $D_{\text{KL}}(\hat{\pi}||\pi)$, where D_{KL} is the KL-divergence, subject to the constraint that the induced trajectories $s_0, s_1, s_2, \dots$ satisfy ϕ . Equivalently, the automaton state sequence $q_0, q_1, \dots$ induced in the product MDP $\mathcal{M}_{\mathcal{A}_\phi}$ intersects infinitely often with the accepting states F . This optimal policy has a closed-form solution given in Equation 1 [53], whose derivation we provide in Appendix A.

$$\hat{\pi}(a_{t:t+H}|s_t, \phi) \propto \pi(a_{t:t+H}|s_t)P(\phi|s_t, q_t, a_{t:t+H}) \quad (1)$$

In Equation 1, $P(\phi \mid s_t, q_t, a_{t:t+H})$ denotes the probability that, after executing chunk $a_{t:t+H}$ from state $\langle s_t, q_t \rangle$ and continuing under π thereafter, the resulting trajectory satisfies ϕ . Based on this factorization, sampling from $\hat{\pi}$ reduces to estimating and incorporating $P(\phi \mid s_t, q_t, a_{t:t+H})$, or its score function, when sampling actions from π [43].

4 hint²

In this section, we present `hint2`, a method for guiding pretrained diffusion policies toward LTL satisfaction at inference time. As shown in Equation 1, the term $P(\phi \mid s_t, q_t, a_{t:t+H})$ captures how the current action chunk affects future satisfaction of the specification ϕ , and is therefore the central quantity needed for LTL guidance. Directly modeling this probability is infeasible, since satisfaction depends on the entire future trajectory induced by the policy and long-horizon prediction quickly suffers from compounding error. `hint2` addresses this by approximating the score $\nabla \log P(\phi \mid s_t, q_t, a_{t:t+H})$ with the gradients of two guidance objectives at different abstraction levels.

High-Level Guidance. The natural abstraction for long-horizon LTL guidance is given by the atomic propositions underlying ϕ . Crucially, proposition values evolve discretely, so repeated labels can be collapsed into single elements that represent long segments of the underlying trajectory. At inference time, this abstraction enables the specification of arbitrary LTL formulas over the same atomic propositions. For a broad class of such specifications, this abstraction is exact: removing duplicated labels has *no impact* on satisfaction for formulas that are “stutter-invariant” with respect to the feasible labels in the environment [54].

Definition 1. For an MDP $\mathcal{M}$, atomic proposition set AP , and labeling function $L : S \rightarrow 2^{AP}$, let $\Sigma_{\mathcal{M}} = \{L(s) \mid s \in S\} \subseteq 2^{AP}$ be the set of labels reachable in $\mathcal{M}$. Let $\mathcal{A}_{\phi} = (Q, 2^{AP}, \delta, q_0, F)$ be the DBA for an LTL formula ϕ . We say ϕ is *stutter invariant* relative to $\mathcal{M}$ if $\delta(\delta(q, \sigma), \sigma) = \delta(q, \sigma)$ for all $q \in Q$ and $\sigma \in \Sigma_{\mathcal{M}}$.

Our *high-level world model* $f^{\text{hi}} : S \times A^H \rightarrow [0, 1]^{N \times |AP|}$ predicts marginal per-proposition probability vectors $\ell_1, \ell_2, \dots, \ell_N$ for the next N distinct labels in the trajectory, given the current state and a short-horizon action chunk. We show how this high-level world model can be used to exactly compute a distribution over future automaton states and derive a tractable guidance objective capable of steering action chunks toward progress through the automaton.

Proposition 1. Let ϕ be an LTL formula with DBA $\mathcal{A}_{\phi} = (Q, 2^{AP}, \delta, q_0, F)$ that is *stutter-invariant* relative to $\mathcal{M}$. Let $\ell_1, \dots, \ell_N \in [0, 1]^{|AP|}$ be a predicted sequence of proposition probabilities, with associated symbol probabilities (assuming labels are mutually independent) and stochastic transition matrices given in Equations 2a and 2b.

$$P_k(\sigma) = \prod_{a \in \sigma} \ell_k(a) \prod_{a \in AP \setminus \sigma} (1 - \ell_k(a)) \quad (2a) \quad M_k(q, q') = \sum_{\sigma \in \Sigma_{\mathcal{M}} : \delta(q, \sigma) = q'} P_k(\sigma) \quad (2b)$$

Then for any initial automaton state $q_t \in Q$ and any segment durations $T_1, \dots, T_N \geq 1$, the distribution over automaton states after N label segments is independent of the segment durations $T_1, \dots, T_N$ and given exactly by $\alpha_N = M_N M_{N-1} \dots M_1 \alpha_0$, where α_0 is the unit vector for q_t .

Proof Sketch. Stutter-invariance (Definition 1) implies that for any $\sigma \in \Sigma_{\mathcal{M}}$, repeating label σ for $T_k \geq 1$ consecutive steps yields the same automaton state as observing it once, since $\delta(\delta(q, \sigma), \sigma) = \delta(q, \sigma)$ for all $q \in Q$. The automaton state after segment k therefore depends only on the segment label, not its duration. Assuming propositions are independent at each step, $P_k(\sigma)$ in Equation 2a is the exact probability of observing symbol σ at segment k . The distribution over automaton states then evolves by $\alpha_k^T = \alpha_{k-1}^T M_k$, and chaining across all N segments gives the result. $\square$

Prior work has shown that satisfaction of ϕ can be ensured by repeatedly enforcing progress toward the accepting states F over a receding horizon, but remains limited to finite discrete systems [30]. We extend this idea to enable LTL guidance of learned policies in stochastic settings by repeatedly maximizing the *expected cumulative automaton potential* $\sum_{k=1}^N v^\top \alpha_k$ over the high-level world model horizon. This is made possible using the intermediate automaton-state distributions $\alpha_1, \dots, \alpha_N$, computed by the recursion $\alpha_k = M_k \alpha_{k-1}$ from Proposition 1, which we prove in Appendix B. Using the automaton potential vector v (Section 3), the cumulative potential provides guidance for

any horizon N as long as there is any expected automaton progress induced by the high-level world model prediction. By backpropagating the cumulative potential through the high-level world model $f^{\text{hi}}(s_t, a_{t:t+H})$ with respect to $a_{t:t+H}$, we obtain a tractable signal that can guide a short-horizon action chunk toward long-horizon LTL satisfaction, given by the high-level term in Equation 3.

Low-Level Guidance. While the high-level world model enables guidance for satisfying entire LTL specifications, including the long-term obligations they express, it does so at the coarse abstraction level of the labels. Their boolean semantics provide a less expressive guidance signal that does not admit an adjustable constraint threshold, which is particularly desirable for hard safety constraint satisfaction. We address this by deriving a second guidance term directly targeting the safety component ϕ_s isolated from ϕ by the decomposition of LTL into safety and liveness components [28]. As safety constraints correspond to avoiding bad prefixes, the robustness (Section 3) of short-horizon trajectories induced by $a_{t:t+H}$ with respect to ϕ_s provides a dense, adjustable guidance signal. We use a *low-level world model* $f^{\text{lo}} : S \times A \rightarrow S$ to predict the trajectory $\tau = (s_t, s_{t+1}, \dots, s_{t+H})$ over which we evaluate the robustness $\rho(\tau, \phi_s)$. The gradient of this robustness with respect to $a_{t:t+H}$, scaled by $\lambda > 0$, yields the low-level guidance signal for enforcing safety in Equation 3.

$$\nabla \log P(\phi \mid s_t, q_t, a_{t:t+H}) \approx \underbrace{\nabla \log \sum_{k=1}^N v^T \alpha_k}_{\text{High-Level}} + \underbrace{\lambda \nabla \rho(f^{\text{lo}}(s_t, a_{t:t+H}), \phi_s)}_{\text{Low-Level}} \quad (3)$$

5 Experimental Results

We evaluate `hint`² across a series of experiments designed to match the settings of existing inference-time steering methods for diffusion policies, while progressively increasing the complexity of both the domain and the specifications. We first isolate long-horizon LTL guidance in a 2D domain, then evaluate guidance in a manipulation environment, and finally train a real-world diffusion policy to demonstrate that the guidance capabilities of `hint`² transfer beyond simulation.

5.1 Temporal-Logic Guided Diffusion

In our first experiment, we compare against LTLDoG [18] and TeLoGraF [15], prior methods that aim to execute general LTL specifications. We conduct this comparison in the 2D Toy Squares domain shown in Fig. 2, where a point agent moves between four colored target regions inducing the propositions `at_green`, `at_red`, `at_yellow`, and `at_blue`. All methods are trained on the same set of demonstrations, in which the agent reaches each target region from a randomized initial state. `hint`² uses an unconditioned multimodal diffusion policy that predicts 8-step action chunks, while the baselines generate state-action trajectories of length 64 for TeLoGraF, and 128 for LTLDoG. For both baselines, we report *planning* satisfaction, measured on the generated state trajectory, and *rollout* satisfaction, measured after executing the corresponding actions in the environment. We evaluate each method on LTL specifications with increasing *automaton distance* from the initial state to an accepting state. An automaton distance of one corresponds to $F \text{ at_blue}$; each additional unit appends one eventual goal from the sequence `at_yellow`, `at_green`, `at_red`, `at_blue`.

Fig. 2 exposes limitations of both baselines. Since the training data contains only single-eventuality demonstrations, TeLoGraF is limited to automaton distance one. Collecting demonstrations for longer specifications becomes intractable, as this scales combinatorially with task horizon. LTLDoG faces a similar issue as satisfying longer LTL formulas requires behavior far beyond the single-target trajectories seen during training, causing its performance to degrade quickly. Beyond suffering from limited training data, full-trajectory generation with a fixed horizon is poorly matched to general LTL guidance, where the required rollout length varies with formula complexity. `hint`² overcomes these limitations and achieves 100% satisfaction across all automaton distances. Rather than generating a satisfying trajectory in one shot, our method repeatedly uses the high-level world model to select 8-step action chunks that progress toward the accepting state of the automaton. This keeps action sampling short-horizon while preserving long-horizon temporal progress, allowing `hint`² to compose behaviors that were never demonstrated as complete trajectories. Appendix C shows that `hint`² also handles richer liveness patterns and safety constraints in this domain.

This comparison shows how `hint`² is capable of handling general LTL specifications much more naturally than existing methods. Since our method simply guides the next action chunk of a diffusion

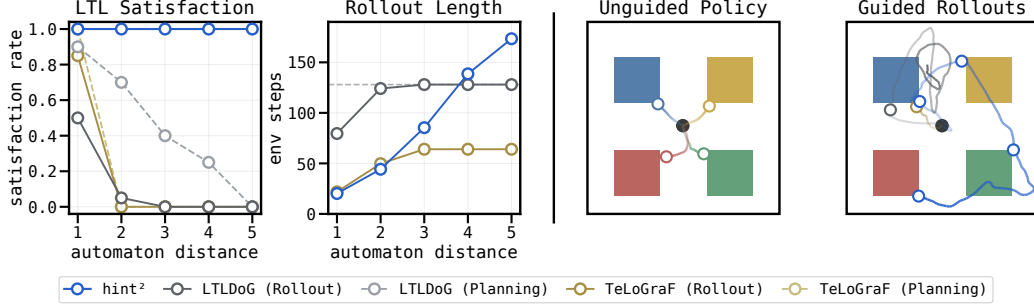


Figure 2: **Comparison of LTL-guided diffusion methods.** hint^2 maintains 100% LTL satisfaction across increasing automaton distances by steering a short-horizon diffusion policy, while the full-trajectory generation used by baselines becomes unreliable and horizon-limited.

policy, it is easily deployable in complex domains. Existing LTL guidance approaches no longer serve as practical baselines in this setting, as the data required for general LTL satisfaction within a single trajectory is infeasible to collect.

5.2 Guidance in a Complex Environment

Our second experiment examines the capabilities of hint^2 in the CALVIN environment [25]. In this setup, a robot interacts with elements of a tabletop scene, including a button, a switch, a drawer, and a sliding door. We train an unconditioned diffusion policy on six hours of human play data in the CALVIN-D dataset. As depicted in Fig. 1, this policy may proceed toward any of the learned behaviors. For the high-level world model, we define the label state over the propositions derived from the behaviors `button_on`, `button_off`, `switch_on`, `switch_off`, `drawer_open`, `drawer_close`, `door_left`, and `door_right`. Further implementation details are provided in Appendix D.

Inference-time Goal Selection. We first compare hint^2 to existing methods that steer unconditioned diffusion policies toward simpler objectives than full LTL specifications. *DynaGuide* [16] steers the policy toward a goal observation using a learned latent dynamics model, while *ITPS* [44] uses a 3D position specified at runtime. In CALVIN, both objectives can be used to select individual tabletop behaviors by steering toward the final observation or end-effector position associated with the desired behavior. Such goal selection corresponds to a small subset of our LTL guidance, namely single-eventuality formulas such as $F \text{ button_on}$. We therefore compare all inference-time diffusion steering methods on single-behavior steering before moving to richer LTL specifications.

Fig. 3 shows that hint^2 substantially improves single-behavior selection. Across the eight evaluated behaviors, hint^2 consistently selects the desired behavior from the base policy, achieving 91% average success and outperforming both baselines. This improvement comes from the high-level world model learning which action chunks are likely to lead to each future behavior, allowing guidance to reliably bias sampling toward the desired outcome. In addition to improved guidance, hint^2 also uses a more accessible runtime interface. Goal images or hand-specified 3D positions can be inconvenient to provide at runtime, whereas symbolic propositions can be specified directly by name without requiring an external grounding input. More importantly, LTL can express objectives that go far beyond selecting one behavior, including complex instructions and safety constraints. Goal-

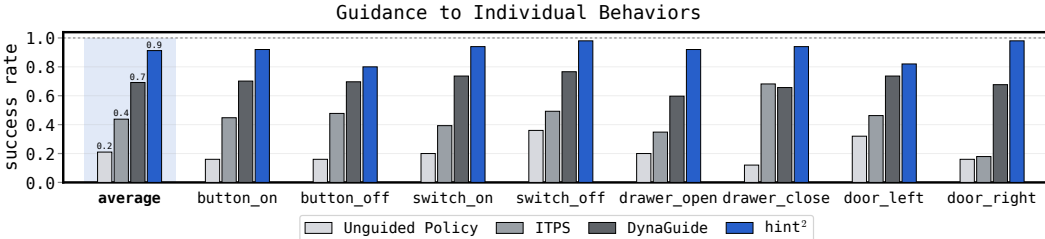


Figure 3: **Inference-time steering for single CALVIN behaviors.** By using single-eventuality LTL objectives, hint^2 consistently selects desired behaviors from an unconditioned multimodal diffusion policy, outperforming DynaGuide and ITPS in their intended setting.

image and position-based steering do not provide an interface for specifying such structure, so we next evaluate hint^2 on richer LTL specifications beyond the scope of these baselines.

Expressive Guidance Capabilities. We now investigate the broader promise of hint^2 : executing complex instructions specified at runtime. Since language-conditioned robot policies have made the most progress toward this goal and use a comparable interface, we compare hint^2 against *FLOWER* [26], a leading VLA on CALVIN, as well as variants augmented with an LLM planner or GPC-based [17] safety guidance. We test both approaches on a variety of complex instructions, including selection among multiple behaviors, unordered, conditional, branched, and chained execution of task sequences, cyclic repetition, as well as safety-constrained variants such as completing a behavior while avoiding an unsafe region, maintaining a desired end-effector angle, or keeping the gripper open. For each, we derive an LTL specification and a corresponding language command, e.g., $\mathbf{F} \text{ drawer_close} \wedge \mathbf{G} \text{ gripper_open}$ and “Close the drawer while keeping the gripper open.” The full list of instructions is provided in Appendix D.2.

Fig. 4 shows that hint^2 achieves near-perfect success across all evaluated specifications. **Liveness:** While *FLOWER* reliably executes individual CALVIN behaviors, it is not capable of completing longer language instructions. This is because the training data contains only primitive commands and learning does not extrapolate to longer-horizon compositions. Adding an open-loop LLM planner improves performance by decomposing the language command into primitive CALVIN skills. For branched and cyclic, this approach still falls short as they require tracking progress through the instruction rather than following a fixed one-shot plan, which highlights the necessity of action selection based on the execution history. Providing the LLM with closed-loop access to the completion history overcomes that problem and achieves the same performance as hint^2 . However, this leads to substantially higher inference cost. By contrast, hint^2 obtains the same progress-tracking behavior directly from the automaton state and lightweight label world model. **Safety:** Language-conditioned policies lack a reliable mechanism to enforce runtime safety constraints not represented during training, so *FLOWER* completes the task without accounting for the constraint. The GPC variant improves safety slightly by sampling multiple action chunks and ranking them using our low-level robustness score. hint^2 instead enforces safety through the low-level component of its guidance score, actively optimizing action chunks away from predicted violations rather than passively selecting the best among sampled candidates. Together, these results demonstrate how hint^2 can elegantly complete challenging inference-time instructions with both long-horizon liveness and safety constraints by handling each aspect at its natural abstraction level.

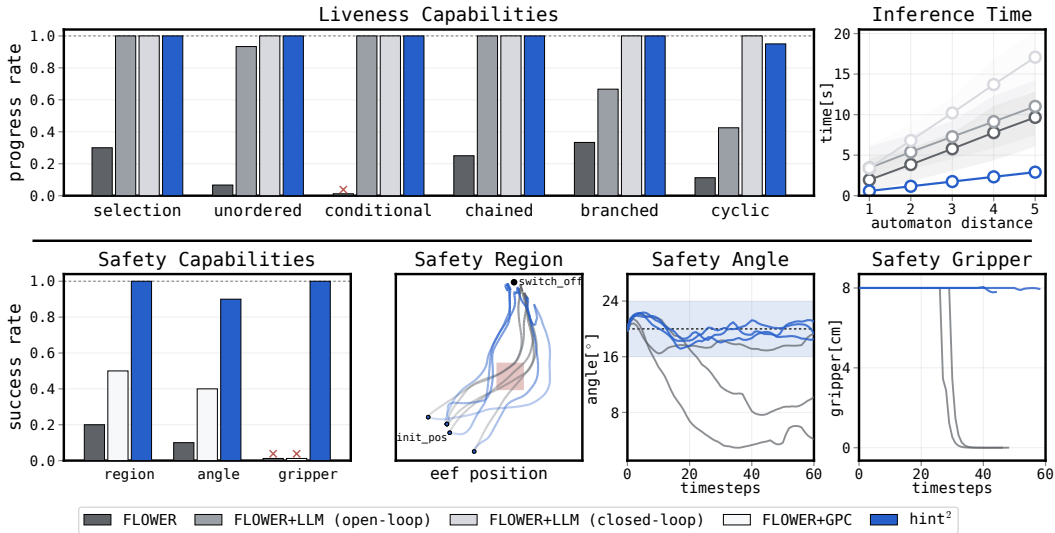


Figure 4: **Comparison on expressive temporal objectives.** hint^2 enables complex inference-time LTL guidance in a high-dimensional manipulation domain, satisfying long-horizon liveness and runtime safety specifications beyond the native capabilities of language-conditioned baselines.

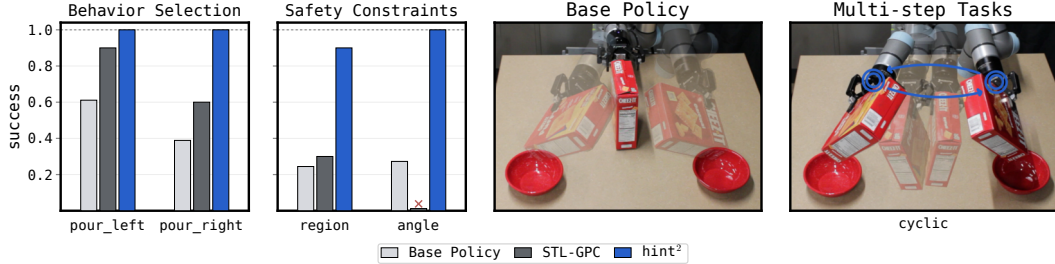


Figure 5: **Real World Experiments.** We show that hint^2 can complete complex real-world instructions, including runtime safety constraints and cyclic behavior, and demonstrate that our guidance signal is fundamentally better matched to short-horizon policies than STL robustness alone.

5.3 Real-world Guidance

Finally, we evaluate whether hint^2 extends beyond simulation and provides effective guidance in a real-world setting. To obtain a multimodal behavior prior, we collect 130 demonstrations in which the robot picks up a box of Cheez-Its, pours into the left bowl, the right bowl, or both bowls, and then places the box back down. We train the unconditioned diffusion policy and both world models on these demonstrations, yielding a policy that can freely transition between behaviors corresponding to the high-level propositions: `box_grabbed`, `pour_left`, and `pour_right`. We evaluate single-behavior mode selection with $F_{\text{pour_left}}$ and $F_{\text{pour_right}}$, as well as harder specifications that require avoiding an unsafe region (`region`), keeping the Cheez-Its upright until the robot is near the target bowl (`angle`), and repeatedly alternating between pouring left and right (`cyclic`). We compare hint^2 against *STL-GPC*, a variant of GPC [17] that samples candidate action chunks and executes the one with the highest STL robustness under the learned world-model rollout.

The real-world results in Fig. 5 show that hint^2 successfully completes all evaluated inference-time instructions. *STL-GPC* improves mode selection, but does not provide the same guidance quality as hint^2 , since committing to the mode that reaches the target bowl often happens before the short-horizon robustness signal becomes informative. On the safety tasks, *STL-GPC* can even perform worse than the base policy because the robustness value must simultaneously capture eventual goal selection and safety avoidance. This creates a brittle guidance signal and can even introduce conflicts, as in `angle`, where pouring robustness encourages tilting the box while the safety constraint requires keeping it upright until the robot approaches the bowl. hint^2 overcomes these limitations by decomposing guidance across abstraction levels and using STL robustness only for local safety guidance, where it is most effective. This enables hint^2 to execute challenging real-world instructions such as cyclic behavior and runtime safety constraints.

6 Conclusion

In this work, we present hint^2 and show that it is possible to guide pretrained diffusion policies in high-dimensional domains to satisfy significantly more complex LTL constraints than those supported by state-of-the-art techniques. Crucial to our method’s success is the prediction of action-chunk consequences at two abstraction levels, in particular predicting at a higher abstraction level that both *captures task-relevant state information* and *evolves on a slower timescale*. We hope this result stimulates further research into world models beyond learning immediate action results and toward learning the dynamics of the world at various abstraction levels and timescales.

Limitations and Future Work. Our method uses a fixed set of atomic propositions to obtain a suitable high-level abstraction. In future work, we plan to investigate methods that learn such discrete, slow-evolving features of the world automatically [55], and use predictions in these latent features to achieve an even more general form of guidance. For LTL guidance specifically, we only show our method obtains an exact guidance signal for constraints that are stutter-invariant relative to the MDP $\mathcal{M}$. We intend to explore supporting more general LTL constraints, including those with non-deterministic automata representations, and extending our approach to multi-agent settings [56].

Acknowledgments

This work was supported by a scholarship of the German Academic Exchange Service (DAAD).

References

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [2] M. Natarajan, E. Seraj, B. Altundas, R. Paleja, S. Ye, L. Chen, R. Jensen, K. C. Chang, and M. Gombolay. Human-robot teaming: grand challenges. *Current Robotics Reports*, 4(3): 81–100, 2023.
- [3] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [4] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.
- [5] S. Zhang, Z. Xu, P. Liu, X. Yu, Y. Li, Q. Gao, Z. Fei, Z. Yin, Z. Wu, Y.-G. Jiang, et al. VLABench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11142–11152, 2025.
- [6] Y. Fan, P. Ding, S. Bai, X. Tong, Y. Zhu, H. Lu, F. Dai, W. Zhao, Y. Liu, S. Huang, et al. Long-VLA: Unleashing long-horizon capability of vision language action model for robot manipulation. *arXiv preprint arXiv:2508.19958*, 2025.
- [7] B. Zhang, Y. Zhang, J. Ji, Y. Lei, J. Dai, Y. Chen, and Y. Yang. SafeVLA: Towards safety alignment of vision-language-action model via constrained learning. *Advances in Neural Information Processing Systems*, 38:153335–153373, 2025.
- [8] S. Hu, Z. Liu, S. Liu, J. Cen, Z. Meng, and X. He. VLSA: Vision-language-action models with plug-and-play safety constraint layer. *arXiv preprint arXiv:2512.11891*, 2025.
- [9] A. Mandlekar, D. Xu, R. Martín-Martín, S. Savarese, and L. Fei-Fei. Learning to generalize across long-horizon tasks from human demonstrations. *arXiv preprint arXiv:2003.06085*, 2020.
- [10] H. Kress-Gazit, G. E. Fainekos, and G. J. Pappas. Temporal-logic-based reactive mission and motion planning. *IEEE transactions on robotics*, 25(6):1370–1381, 2009.
- [11] C. Baier and J.-P. Katoen. *Principles of model checking*. MIT press, 2008.
- [12] O. Maler and D. Nickovic. Monitoring temporal properties of continuous signals. In Y. Lakhnech and S. Yovine, editors, *Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems (FTRTFT)*, volume 3253 of *Lecture Notes in Computer Science*, pages 152–166, Berlin, Heidelberg, 2004. Springer. ISBN 978-3-540-30206-3. doi: [10.1007/978-3-540-30206-3_12](https://doi.org/10.1007/978-3-540-30206-3_12).
- [13] J. X. Liu, Z. Yang, B. Schornstein, S. Liang, I. Idrees, S. Tellex, and A. Shah. Lang2LTL: Translating natural language commands to temporal specification with large language models. In *Workshop on Language and Robotics at CoRL 2022*, 2022.
- [14] J. X. Liu, A. Shah, G. Konidaris, S. Tellex, and D. Paulius. Lang2LTL-2: Grounding spatiotemporal navigation commands using large language and vision-language models. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2325–2332. IEEE, 2024.

- [15] Y. Meng and C. Fan. TeLoGraF: Temporal logic planning via graph-encoded flow matching. *arXiv preprint arXiv:2505.00562*, 2025.
- [16] M. Du and S. Song. DynaGuide: Steering diffusion policies with active dynamic guidance. *Advances in Neural Information Processing Systems*, 38:44192–44221, 2025.
- [17] H. Qi, H. Yin, A. Zhu, Y. Du, and H. Yang. Inference-time enhancement of generative robot policies via predictive world modeling. *IEEE Robotics and Automation Letters*, 11(5):5534–5541, 2026. doi:10.1109/LRA.2026.3673995.
- [18] Z. Feng, H. Luan, P. Goyal, and H. Soh. LTLDoG: Satisfying temporally-extended symbolic constraints for safe diffusion-based planning. *IEEE Robotics and Automation Letters*, 9:8571–8578, 10 2024. doi:10.1109/lra.2024.3443501. URL <https://doi.org/10.1109/lra.2024.3443501>.
- [19] Y. Meng and C. Fan. Diverse controllable diffusion policy with signal temporal logic. *IEEE Robotics and Automation Letters*, 9:8354–8361, 10 2024. doi:10.1109/lra.2024.3444668. URL <https://doi.org/10.1109/lra.2024.3444668>.
- [20] M. Janner, J. Fu, M. Zhang, and S. Levine. When to trust your model: Model-based policy optimization. *Advances in neural information processing systems*, 32, 2019.
- [21] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- [22] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- [23] P. Kapoor, A. Ganlath, M. Clifford, C. Liu, S. Scherer, and E. Kang. SafeDec: Constrained decoding for safe autoregressive generalist robot policies, 2026. URL <https://openreview.net/forum?id=dL07MhVbbB>.
- [24] M. Zoellner, A. Manganaris, and R. Paleja. Temporal logic guidance for action-only diffusion policies with world models. *arXiv preprint arXiv:2606.22729*, 2026.
- [25] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334, 2022.
- [26] M. Reuss, H. Zhou, M. Rühle, Ö. E. Yağmurlu, F. Otto, and R. Lioutikov. FLOWER: Democratizing generalist robot policies with efficient vision-language-action flow policies. *arXiv preprint arXiv:2509.04996*, 2025.
- [27] C. Belta and S. Sadraddini. Formal methods for control synthesis: An optimization perspective. *Annual Review of Control, Robotics, and Autonomous Systems*, 2:115–140, 5 2019. doi:10.1146/annurev-control-053018-023717. URL <https://doi.org/10.1146/annurev-control-053018-023717>.
- [28] B. Alpern and F. B. Schneider. Recognizing safety and liveness. *Distributed Computing*, 2:117–126, 9 1987. doi:10.1007/bf01782772. URL <https://doi.org/10.1007/bf01782772>.
- [29] T. Wongpiromsarn, U. Topcu, and R. M. Murray. Receding horizon control for temporal logic specifications. In *Proceedings of the 13th ACM international conference on Hybrid systems: computation and control*, pages 101–110, 2010.
- [30] X. Ding, M. Lazar, and C. Belta. Ltl receding horizon control for finite deterministic systems. *Automatica*, 50:399–408, 2 2014. doi:10.1016/j.automatica.2013.11.030. URL <https://doi.org/10.1016/j.automatica.2013.11.030>.

- [31] D. Sadigh, E. S. Kim, S. Coogan, S. S. Sastry, and S. A. Seshia. A learning based approach to control synthesis of markov decision processes for linear temporal logic specifications. In *53rd IEEE Conference on Decision and Control*, pages 1091–1096. IEEE, 2014.
- [32] M. Hasanbeig, A. Abate, and D. Kroening. Logically-constrained reinforcement learning. *arXiv preprint arXiv:1801.08099*, 2018.
- [33] R. Toro Icarte, T. Q. Klassen, R. Valenzano, and S. A. McIlraith. Reward machines: Exploiting reward function structure in reinforcement learning. *Journal of Artificial Intelligence Research*, 73:173–208, Jan. 2022. ISSN 1076-9757. doi:10.1613/jair.1.12440.
- [34] K. Jothimurugan, S. Bansal, O. Bastani, and R. Alur. Compositional reinforcement learning from logical specifications. *Advances in Neural Information Processing Systems*, 34:10026–10039, 2021.
- [35] A. Donzé and O. Maler. Robust satisfaction of temporal logic over real-valued signals. In *International conference on formal modeling and analysis of timed systems*, pages 92–106. Springer, 2010.
- [36] K. Leung, N. Aréchiga, and M. Pavone. Backpropagation through signal temporal logic specifications: Infusing logical structure into gradient-based methods. *The International Journal of Robotics Research*, 42(6):356–370, 2023.
- [37] D. Aksaray, A. Jones, Z. Kong, M. Schwager, and C. Belta. Q-learning for robust satisfaction of signal temporal logic specifications. In *2016 IEEE 55th Conference on Decision and Control (CDC)*, pages 6565–6570. IEEE, 2016.
- [38] P. Kapoor, A. Balakrishnan, and J. V. Deshmukh. Model-based reinforcement learning from signal temporal logic specifications. *arXiv preprint arXiv:2011.04950*, 2020.
- [39] Y. Meng and C. Fan. Signal temporal logic neural predictive control. *IEEE Robotics and Automation Letters*, 8(11):7719–7726, 2023.
- [40] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [41] P. Kapoor, S. Vemprala, and A. Kapoor. Logically constrained robotics transformers for enhanced perception-action planning. *arXiv preprint arXiv:2408.05336*, 2024.
- [42] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- [43] A. Bansal, H.-M. Chu, A. Schwarzschild, R. Sengupta, M. Goldblum, J. Geiping, and T. Goldstein. Universal guidance for diffusion models. In *International Conference on Learning Representations*, volume 2024, pages 51304–51323, 2024.
- [44] Y. Wang, L. Wang, Y. Du, B. Sundaralingam, X. Yang, Y.-W. Chao, C. Pérez-D’Arpino, D. Fox, and J. Shah. Inference-time policy steering through human interactions. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 15626–15633. IEEE, 2025.
- [45] H. Ma, S. Bodmer, A. Carron, M. Zeilinger, and M. Muehlebach. Constraint-aware diffusion guidance for robotics: Real-time obstacle avoidance for autonomous racing. *arXiv preprint arXiv:2505.13131*, 2025.
- [46] M. Janner, Y. Du, J. Tenenbaum, and S. Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning*, 2022.

- [47] Z. Zhong, D. Rempe, D. Xu, Y. Chen, S. Veer, T. Che, B. Ray, and M. Pavone. Guided conditional diffusion for controllable traffic simulation. In *2023 IEEE international conference on robotics and automation (ICRA)*, pages 3560–3566. IEEE, 2023.
- [48] J. Huang, W. Chen, Z. Li, O. Pang, X. Hu, L. Zhang, Y. Hu, Z. Zhang, M. Coates, T. Cao, et al. H-wm: Robotic task and motion planning guided by hierarchical world model. *arXiv preprint arXiv:2602.11291*, 2026.
- [49] W. Zhang, B. Terver, A. Zholus, S. Chitnis, H. Sutaria, M. Assran, R. Balestriero, A. Bar, A. Bardes, Y. LeCun, et al. Hierarchical planning with latent world models. *arXiv preprint arXiv:2604.03208*, 2026.
- [50] N. Hansen, J. SV, V. Sobal, Y. LeCun, X. Wang, and H. Su. Hierarchical world models as visual whole-body humanoid controllers. In *International Conference on Learning Representations*, volume 2025, pages 62175–62195, 2025.
- [51] A. Pnueli. The temporal logic of programs. In *Symposium on Foundations of Computer Science (SFCS)*, pages 46–57, 1977. doi:10.1109/SFCS.1977.32.
- [52] Z. Manna and A. Pnueli. A hierarchy of temporal properties. In *ACM Symposium on Principles of Distributed Computing (PODC)*, pages 377–410, New York, NY, USA, 1990. Association for Computing Machinery. ISBN 089791404X. doi:10.1145/93385.93442. URL <https://doi.org/10.1145/93385.93442>.
- [53] S. Levine. Reinforcement learning and control as probabilistic inference: Tutorial and review, 2018. URL <https://arxiv.org/abs/1805.00909>.
- [54] D. Peled and T. Wilke. Stutter-invariant temporal properties are expressible without the next-time operator. *Information Processing Letters*, 63(5):243–246, 1997. ISSN 0020-0190. doi:10.1016/S0020-0190(97)00133-6. URL <https://www.sciencedirect.com/science/article/pii/S0020019097001336>.
- [55] C. Gumbsch, N. Sajid, G. Martius, and M. V. Butz. Learning hierarchical world models with adaptive temporal abstractions from discrete latent dynamics. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=TjCDNssXKU>.
- [56] T.-H. Hsu, A. Rafieioskouei, and B. Bonakdarpour. HypRL: Reinforcement learning of control policies for hyperproperties. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=lJSAtyx9Uc>.
- [57] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *arXiv preprint arXiv:2108.03298*, 2021.
- [58] B. Wen, W. Yang, J. Kautz, and S. Birchfield. FoundationPose: Unified 6d pose estimation and tracking of novel objects. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 17868–17879, 2024. doi:10.1109/CVPR52733.2024.01692.

A Derivation of Optimal Policy Factorization

In this section, we provide the derivation for the factored form of the LTL-guided policy $\hat{\pi}$ given in Equation 1 of our main paper. For this derivation, we formalize the constraint that trajectories $s_0, s_1, s_2, \dots$ induced by $\hat{\pi}$ satisfy ϕ by constraining the logarithm of the probability of continuing to satisfy ϕ given any current state, automaton state, and action $\langle s, q, a \rangle$, in expectation over possible policies $\hat{\pi}$, to exceed some threshold $\epsilon \in (-\infty, 0]$. The specific value of epsilon can be chosen to enforce satisfaction with probability up to 1, but the functional form of $\hat{\pi}$ does not depend on the specific value of ϵ . The full optimization problem, with the objective of minimizing the KL-divergence from the base policy π and the constraint that $\hat{\pi}$ remains a valid distribution, is given below.

$$\begin{aligned} \min \quad & D_{\text{KL}}(\hat{\pi} \parallel \pi) \\ \text{subject to} \quad & \mathbb{E}_{\hat{\pi}} [\log P(\phi \mid s, q, a)] \geq \epsilon \\ & \int \hat{\pi}(a \mid s, q) da = 1 \end{aligned}$$

The definition for KL-divergence is given in Equation 4.

$$D_{\text{KL}}(\hat{\pi} \parallel \pi) = \int \hat{\pi}(a \mid s, q) \log \frac{\hat{\pi}(a \mid s, q)}{\pi(a \mid s, q)} da \quad (4)$$

The Lagrangian for this optimization problem with multipliers $\beta \geq 0$ and λ is given in Equation 5.

$$\mathcal{L}(\hat{\pi}, \beta, \lambda) = \int \hat{\pi}(a \mid s, q) \log \frac{\hat{\pi}(a \mid s, q)}{\pi(a \mid s, q)} da - \beta (\mathbb{E}_{\hat{\pi}} [\log P(\phi \mid s, q, a)] - \epsilon) - \lambda \left(\int \hat{\pi}(a \mid s, q) da - 1 \right) \quad (5)$$

We can derive the form of the solution by taking the derivative of the Lagrangian with respect to $\hat{\pi}$ and setting it to zero.

$$\frac{\partial \mathcal{L}}{\partial \hat{\pi}(a \mid s, q)} = \log \frac{\hat{\pi}(a \mid s, q)}{\pi(a \mid s, q)} + 1 - \beta \log P(\phi \mid s, q, a) - \lambda = 0 \quad (6)$$

Solving for $\hat{\pi}$ yields Equation 8.

$$\log \hat{\pi}(a \mid s, q) = \log \pi(a \mid s, q) + \beta \log P(\phi \mid s, q, a) + (\lambda - 1) \quad (7)$$

$$\hat{\pi}(a \mid s, q) = \pi(a \mid s, q) P(\phi \mid s, q, a)^\beta e^{\lambda - 1} \quad (8)$$

Therefore, using $\beta = 1$, the corresponding value of λ determining the normalization constant, and noting that $\pi(a \mid s, q) = \pi(a \mid s)$ yields $\hat{\pi}(a \mid s, q) \propto \pi(a \mid s) P(\phi \mid s, q, a)$.

B Proof of Proposition 1

In this section, we restate Proposition 1 from our main paper and provide its full proof. We also give additional support for our approach maximizing expected automaton potential to encourage satisfying the input LTL specification, by analogy to the automaton potential constraint used by Ding et al. [30].

Proposition 1 (restated). Let ϕ be an LTL formula with DBA $\mathcal{A}_\phi = (Q, 2^{AP}, \delta, q_0, F)$ that is stutter-invariant relative to $\mathcal{M}$. Let $\ell_1, \dots, \ell_N \in [0, 1]^{|AP|}$ be a predicted sequence of proposition probabilities, with associated symbol probabilities (assuming labels are mutually independent) and stochastic transition matrices given in Equations 9a and 9b.

$$P_k(\sigma) = \prod_{a \in \sigma} \ell_k(a) \prod_{a \in AP \setminus \sigma} (1 - \ell_k(a)) \quad (9a) \quad M_k(q, q') = \sum_{\sigma \in \Sigma_{\mathcal{M}} : \delta(q, \sigma) = q'} P_k(\sigma) \quad (9b)$$

Then for any initial automaton state $q_t \in Q$ and any segment durations $T_1, \dots, T_N \geq 1$, the distribution over automaton states after N label segments is independent of the segment durations $T_1, \dots, T_N$ and given exactly by $\alpha_N = M_N M_{N-1} \dots M_1 \alpha_0$, where α_0 is the unit vector for q_t .

Proof. First, we claim that for any automaton state $q \in Q$, any label $\sigma \in \Sigma_{\mathcal{M}}$, and any $T \geq 1$, the automaton state after observing σ repeated T times is the same as after observing σ once. Denoting the T -fold application of δ with a repeated label σ by $\delta^T(q, \sigma)$, this can be written as $\delta^T(q, \sigma) = \delta(q, \sigma)$ for all $T \geq 1$. We prove this by induction on T . The base case when $T = 1$ is immediate. The induction step, following from Definition 1 in our main paper, is shown in Equation 10.

$$\delta^T(q, \sigma) = \delta(q, \sigma) \implies \delta^{T+1}(q, \sigma) = \delta(\delta^T(q, \sigma), \sigma) = \delta(\delta(q, \sigma), \sigma) = \delta(q, \sigma) \quad (10)$$

By induction, the automaton state after segment k is entirely determined by the segment label σ_k , regardless of its duration $T_k \geq 1$.

Second, we show $\alpha_k = M_k \alpha_{k-1}$. Let q_k denote the automaton state after segment k , and let $\alpha_k \in [0, 1]^{|Q|}$ denote its distribution. For any $q' \in Q$, the probability of q_k being q' can be related to the probability of each automaton state at the previous iteration (from α_{k-1}) by the law of total probability.

$$P(q_k = q') = \sum_{q \in Q} P(q_k = q' \mid q_{k-1} = q) \cdot P(q_{k-1} = q) \quad (11)$$

$$= \sum_{q \in Q} P(q_k = q' \mid q_{k-1} = q) \cdot (\alpha_{k-1})_q \quad (12)$$

The probability of transitioning to q' coming from q_{k-1} is the probability $P_k(\sigma)$ of observing a label $\sigma \in \Sigma_{\mathcal{M}}$ at step k satisfying $q' = \delta(q_{k-1}, \sigma)$. Therefore, we can rewrite $P(q_k = q' \mid q_{k-1} = q)$ as in Equation 13, which gives each element for the matrix M_k .

$$P(q_k = q' \mid q_{k-1} = q) = \sum_{\substack{\sigma \in \Sigma_{\mathcal{M}} \\ \delta(q, \sigma) = q'}} P_k(\sigma) = M_k(q, q') \quad (13)$$

Under the assumption that propositions are mutually independent at each step, $P_k(\sigma)$ factorizes as given in Equation 9a, which is the exact marginal probability of each symbol under the label probabilities ℓ_k . Substituting Equation 13 into Equation 12 and rewriting in vector form yields the recursion $\alpha_k = M_k \alpha_{k-1}$.

Finally, applying the recursion across all steps $k = 1, \dots, N$ gives $\alpha_N = M_N M_{N-1} \dots M_1 \alpha_0$ and initializing with the unit vector for q_t ($\alpha_0 = e_{q_t}$) gives the result. $\square$

B.1 Justification for Maximizing Expected Automaton Potential

We note that our guidance signal, based on maximizing the expected cumulative automaton potential over the high-level prediction horizon, can be seen as a stochastic extension of the constrained receding horizon controller proposed by Ding et al. [30] for deterministic finite systems. Their controller optimizes a reward over a finite horizon in a product MDP while (when not at an accepting automaton state) enforcing a constraint that closeness to automaton accepting states increases between iterations. With our potential function, this constraint can be written as $v_{q_{N|k}} \geq v_{q_{N|k-1}}$, where $q_{N|k}$ denotes the automaton state after N distinct labels for iteration k . This constraint guarantees repeated visits to F and therefore satisfaction of ϕ [30]. Enforcing this constraint directly is not possible in our stochastic setting for two reasons. First, we have access only to a stochastic approximation of the automaton-level dynamics, so the terminal automaton state $q_{N|k}$ is a random variable rather than a deterministic quantity. Second, even replacing the hard constraint with its expectation $\mathbb{E}[v_{q_{N|k}}] \geq v_{q_{N|k-1}}$ may yield an infeasible problem: if all action chunks assign low but nonzero probability to progress-making transitions, the expected terminal potential may fall below $v_{q_{N|k-1}}$ for every available chunk, despite some probability of progress existing. We therefore replace the hard progress constraint with a soft maximization objective. Maximizing $\sum_{k=1}^N v^T \alpha_k$ encourages persistent increase in $\mathbb{E}[v_{q_{N|k}}]$ across iterations in the same spirit as the Ding et al. constraint, while remaining feasible in all cases. Furthermore, optimizing the cumulative potential rather than the terminal potential $v^T \alpha_N$ alone induces a preference for shorter paths to F : two action chunks reaching F at steps $k_1 < k_2$ respectively contribute $v^T \alpha_{k_1:N} > v^T \alpha_{k_2:N}$ to the sum, so earlier progress is rewarded.

C Toy Squares - Complex Specification

To verify that the Toy Squares results are not limited to simple ordered reachability, we additionally evaluate a richer LTL specification combining unordered liveness, branching, sequencing, and safety:

$$\begin{aligned} \phi_{\text{rich}} = & \mathbf{F} \left((\mathbf{F} \text{red} \wedge \mathbf{F} \text{blue}) \wedge \mathbf{F} \left[\mathbf{F} (\text{yellow} \wedge \mathbf{F} (\text{blue} \wedge \mathbf{F} (\text{green} \wedge \mathbf{F} \text{yellow}))) \right. \right. \\ & \left. \left. \vee \mathbf{F} (\text{green} \wedge \mathbf{F} (\text{blue} \wedge \mathbf{F} (\text{yellow} \wedge \mathbf{F} \text{green}))) \right] \right) \\ & \wedge \mathbf{G} \neg \text{unsafe_regions}. \end{aligned} \quad (14)$$

This specification first requires the agent to reach red and blue in either order. It then branches between two possible sequences: yellow, blue, green, yellow, or green, blue, yellow, green. Finally, the full task must be completed while always avoiding the unsafe regions. This combines the main temporal concepts evaluated throughout the paper, and `hint`² achieves 100% satisfaction, demonstrating that the hierarchical guidance confidently handles complex combinations of liveness and safety constraints in the Toy Squares domain. Figure 6 shows a representative rollout.

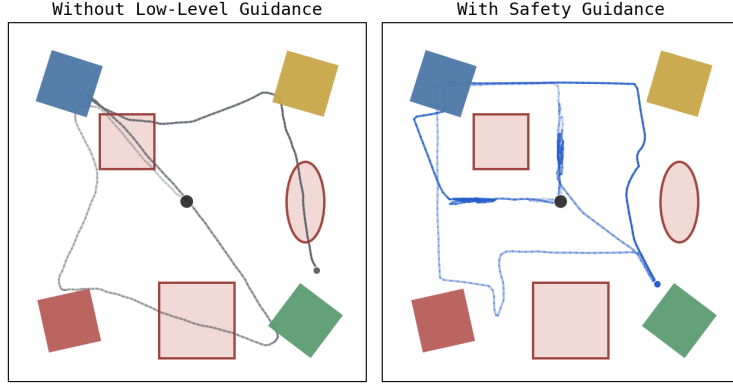


Figure 6: **Additional Toy Squares specification.** `hint`² satisfies a complex LTL task combining unordered and ordered sequencing, branching, and safety constraints.

D CALVIN Implementation

D.1 Model Training

For this set of experiments, we adapt the diffusion-policy implementation from robomimic [57] to the CALVIN [25] environment, following the setup used by DynaGuide [16]. The policy observes proprioception together with wrist and third-person RGB images, and predicts action trajectories with prediction horizon 16 and execution horizon 8. Our world models operate on low-dimensional robot and scene state. All models are trained solely on the CALVIN-D training dataset.

The high-level world model takes the current robot state, scene state, current binary label vector, and an 8-step action chunk as input, and predicts the next label probability vector with prediction horizon $N = 1$. Labels are generated from the CALVIN scene state based on thresholds provided by the environment. The label set consists of `switch_on`, `switch_off`, `button_pressed`, `drawer_open`, `drawer_close`, `door_left`, and `door_right`. The model uses separate MLP encoders for the state, action chunk, and label vector, concatenates the resulting embeddings, and predicts the future binary label vector. We train the high-level world model with binary cross-entropy loss using Adam.

The low-level dynamics world model is a single-step transition model over the robot and scene state. It takes the current state and a single 7D action as input and predicts the normalized state delta. For

rotations, we use a continuous 6D representation. The dynamics model is an MLP with hidden size 512, depth 4, and dropout 0.02. We train the dynamics model with mean-squared error loss on normalized state deltas using Adam.

D.2 Experiments

For the individual guidance experiment, we replicate the articulated-object evaluation setup from DynaGuide, using the same task set, reset configurations, initial-state sampling, and randomization procedure. The tasks `button_on` and `button_off` are both represented using the label `button_pressed`, as they use the same underlying movement.

Task	TL specification	Language command
selection	$\mathbf{F}(\text{door_left} \vee \text{switch_off} \vee \text{button_pressed})$	Move the sliding door to the left, turn off the lightbulb using the switch, or press the button.
unordered	$\mathbf{F} \text{ door_left} \wedge \mathbf{F} \text{ switch_off} \wedge \mathbf{F} \text{ button_pressed}$	Move the sliding door to the left, turn off the lightbulb using the switch, and press the button in any order.
conditional	$\mathbf{F} \text{ drawer_close} \wedge (\neg \text{drawer_close} \mathbf{U} \text{ button_pressed}) \wedge (\neg \text{drawer_close} \mathbf{U} \text{ switch_off})$	Close the drawer, but press the button and turn off the lightbulb using the switch before that.
chained	$\mathbf{F}(\text{button_pressed} \wedge \mathbf{XF}(\text{door_right} \wedge \mathbf{XF}(\text{switch_off} \wedge \mathbf{XF} \text{ drawer_close})))$	Press the button, then move the sliding door to the right, then turn off the lightbulb using the switch, then close the drawer.
branched	$\mathbf{F}(\text{button_pressed} \wedge \mathbf{XF}(\text{drawer_close} \wedge \mathbf{XF} \text{ switch_off})) \vee \mathbf{F}(\text{switch_off} \wedge \mathbf{XF}(\text{drawer_close} \wedge \mathbf{XF} \text{ button_pressed}))$	Press the button or turn off the lightbulb using the switch, then close the drawer, then do the remaining option.
cyclic	$\mathbf{GF}(\text{drawer_open} \wedge \mathbf{XF}(\text{switch_on} \wedge \mathbf{XF}(\text{drawer_close} \wedge \mathbf{XF} \text{ switch_off})))$	Repeatedly open the drawer, turn on the lightbulb using the switch, close the drawer, and turn off the lightbulb using the switch in order.
region	$\mathbf{F} \text{ switch_off} \wedge \mathbf{G} \neg \text{unsafe_region}$	Turn off the lightbulb using the switch while keeping the robot arm out of the unsafe region.
angle	$\mathbf{F} \text{ door_right} \wedge \mathbf{G} \text{ tcp_tilt_20deg}$	Move the sliding door to the right while keeping the TCP tilted near 20 degrees.
gripper	$\mathbf{F} \text{ drawer_close} \wedge \mathbf{G} \text{ gripper_open}$	Close the drawer while keeping the gripper open.

Table 1: **Specifications for expressive guidance.** We pair each TL specification used for `hint`² with the corresponding natural-language command given to the language-conditioned baselines. The first six specifications test task-level temporal structure, while the final three combine task completion with runtime safety constraints.

For the second experiment, we evaluate composite temporal-logic objectives against language-conditioned baselines. Table 1 lists the full set of LTL specifications and corresponding language instructions used in the results. Unlike the DynaGuide comparison, we remove the movable blocks from the tabletop in the scene setup. The blocks are not part of our high-level label set, so removing them keeps the evaluation focused on temporal guidance over the behaviors present in the label set and explains the increase in success from 91% in the single-guidance experiments to 99% in the complex guidance experiments.

To guide the diffusion policy in both setups, we sample 32 candidate 8-action chunks and execute the one with the highest expected cumulative automaton potential. When safety constraints are present, the dynamics model prediction is integrated into the denoising process through gradients of the low-level scoring term, and we perform 10 additional gradient steps on the sampled action chunk after denoising.

E Further Implementation Details

The Toy Squares and real-world experiments use the same implementation structure as CALVIN. In all domains, the base policy is a diffusion policy adapted from robomimic, and the high-level and low-level world models are implemented as MLPs operating on low-dimensional state observations. We adjust only the model sizes to match the complexity of each domain. In the real-world setup, we

obtain the low-dimensional state by using FoundationPose [58] to track the position and orientation of the Cheez-It box.

For cyclic tasks, we terminate evaluation after two full cycles, since true infinite-horizon evaluation is not possible. In the real-world cyclic task, `hint`² achieves 100% success over five runs.

Videos and code will be made available on the project website: <https://anonymous-hint2.github.io/>.